

Package ‘mvPred’

September 30, 2026

Type Package

Title Methods for Handling Missing Values in Linear Modeling

Version 0.1.0

Maintainer Billy Ouattara <ouattarabilly33@hotmail.com>

Description Provides user-friendly methods for handling missing data in regression modeling, including available-case linear regression, multiple imputation, random-forest imputation, and the Tower method. Implemented approaches include chained-equation imputation described by van Buuren and Groothuis-Oudshoorn (2011) <doi:10.18637/jss.v045.i03>, multiple imputation described by Honaker, King and Blackwell (2011) <doi:10.18637/jss.v045.i07>, random-forest imputation described by Stekhoven and Buehlmann (2012) <doi:10.1093/bioinformatics/btr597>, and the Tower method described by Matloff and Mohanty (2023) <<https://CRAN.R-project.org/package=toweranNA>>.

Depends R (>= 4.0.0)

Imports regtools, mice, Amelia, missForest, toweranNA, stats, utils, qeML

License MIT + file LICENSE

URL <https://github.com/matloff/mvPred>

BugReports <https://github.com/matloff/mvPred/issues>

Encoding UTF-8

LazyData true

RoxygenNote 7.3.3

NeedsCompilation no

Author Norman Matloff [aut],
Cynthia Mascarenhas [aut],
Billy Ouattara [aut, cre],
Aleky Veluri [aut]

Repository CRAN

Date/Publication 2026-09-30 09:10:09 UTC

Contents

auto_mpg	2
bootstrap	3
compCases	5
lm_ac	5
lm_compare	7
lm_prefill	9
lm_tower	11
qeMLna	13
Index	15

auto_mpg	<i>Datasets Included in mvPred</i>
----------	------------------------------------

Description

Datasets distributed with **mvPred** for examples, experiments, and package demonstrations.

Usage

```
data("auto-mpg")
data(english)
data(NHkids)
```

Format

auto_mpg: a data frame based on the Auto MPG data, including fuel-efficiency and vehicle-characteristic variables.

english: a data frame used for predictive modeling examples involving vocabulary outcomes and demographic predictors.

NHkids: a data frame used for predictive modeling examples involving child weight, height, and age in months.

Details

These datasets are included to support examples and methodological comparisons within the package.

Examples

```
data("auto-mpg", package = "mvPred")
data("english", package = "mvPred")
data("NHkids", package = "mvPred")
```

Description

Run k-fold cross-validation for regression while applying one of the package's missing-data handling strategies. Depending on the selected method, the function can fit complete-case, available-case, PREFILL, or TOWER-based models and return fold-level summaries.

Usage

```
bootstrap(
  data,
  yName,
  k = 5,
  task = c("regression", "classification"),
  method = c("AC", "CC", "PREFILL", "TOWER"),
  threshold = 0.5,
  seed = 42,
  test_split = 0.2,
  impute_method = c("mice", "amelia", "missforest", "complete"),
  mice_method = NULL,
  m = 5,
  use_dummies = FALSE,
  tower_regFtnName = "lm",
  tower_opts = list(),
  tower_scaling = NULL,
  tower_yesYVal = NULL,
  ...
)
```

Arguments

<code>data</code>	A data frame containing predictors and the response variable.
<code>yName</code>	The name of the response column in data.
<code>k</code>	Number of folds used for cross-validation.
<code>task</code>	Whether to run "regression" or "classification" metrics.
<code>method</code>	Missing-data handling strategy: "AC", "CC", "PREFILL", or "TOWER".
<code>threshold</code>	Classification threshold used when <code>task = "classification"</code> .
<code>seed</code>	Random seed used for split reproducibility.
<code>test_split</code>	Proportion of the full dataset reserved for the final holdout test set.
<code>impute_method</code>	Imputation method used when <code>method = "PREFILL"</code> .
<code>mice_method</code>	Optional mice method specification for PREFILL runs.
<code>m</code>	Number of multiple imputations used by applicable PREFILL methods.

<code>use_dummies</code>	Logical indicating whether categorical variables should be converted to dummies for Amelia-based PREFILL runs.
<code>tower_regFtnName</code>	Regression routine passed to the TOWER method.
<code>tower_opts</code>	Optional list of control settings passed to TOWER.
<code>tower_scaling</code>	Optional scaling object passed to TOWER.
<code>tower_yesYVal</code>	Optional flag passed to TOWER.
<code>...</code>	Additional arguments forwarded to the selected fitting routine.

Details

The function first optionally creates a holdout split controlled by `test_split`. Cross-validation is then run on the training portion using the requested missing-data handling method. For PREFILL runs, the supported imputation methods are `mice`, `Amelia`, `missForest`, and complete-case preprocessing.

Value

A list containing fold-level performance summaries and fitted model objects. Depending on the selected task, the returned metrics correspond to regression or classification performance.

Common returned components include:

- `fold_models`: list of fitted models, one per cross-validation fold.
- `final_model`: fitted model from the final training split when `test_split > 0`.
- `test_results`: performance on the final holdout test set when `test_split > 0`.
- `training_missingness_by_fold`: missingness tables for each training fold.
- `training_missingness_average`: average missingness across training folds.

Examples

```
data("auto-mpg", package = "mvPred")

df <- auto_mpg
df$car_name <- NULL

for (nm in names(df)) {
  suppressWarnings(df[[nm]] <- as.numeric(df[[nm]]))
}

res <- bootstrap(
  data = df,
  yName = "mpg",
  k = 3,
  task = "regression",
  method = "CC"
)

res$RMSE_mean
```

compCases

*Extract Complete Cases from a Data Frame***Description**

Remove rows containing missing values and report which rows were retained or discarded.

Usage

```
compCases(data)
```

Arguments

`data` A data frame or matrix that may contain missing values.

Value

A list with components:

- `intactData`: rows of data with no missing values.
- `intactRows`: logical vector indicating complete rows.
- `nonintactRows`: integer indices of rows containing at least one missing value.

Examples

```
df <- data.frame(x = c(1, NA, 3), y = c(2, 4, 6))
compCases(df)
```

lm_ac

*Linear Regression Using Available Cases Methods***Description**

This suite of functions implements linear regression using the Available Cases (pairwise deletion) approach. It makes use of the available cases in the dataset to fit the model and make predictions rather than deleting any rows with missing values. The main function `lm_ac` fits the model even when the predictors or response contain missing values. The helper functions compute matrix products using only complete pairs of observations. S3 methods are included for summary and prediction.

Usage

```
lm_ac(data, yName, holdout = NULL, ...)

ac_mul(A)

ac_vec(X, y)

## S3 method for class 'lm_ac'
summary(object, ...)

## S3 method for class 'lm_ac'
predict(object, newdata, ...)
```

Arguments

<code>data</code>	A data.frame containing predictors and the response variable.
<code>yName</code>	The name of the response column in data.
<code>holdout</code>	Optional holdout specification used to define testing rows. This may be a logical vector of length <code>nrow(data)</code> or a numeric vector of row indices. If <code>NULL</code> , all rows are used for fitting.
<code>...</code>	Additional arguments passed to <code>model.frame</code> in <code>lm_ac</code> .
<code>A</code>	A numeric matrix or data frame to be used in <code>ac_mul</code> to compute ATA.
<code>X</code>	A numeric matrix of predictors for <code>ac_vec</code> .
<code>y</code>	A numeric vector representing the response variable for <code>ac_vec</code> .
<code>object</code>	An object of class <code>lm_ac</code> for S3 methods which is returned from calling <code>lm_ac</code> .
<code>newdata</code>	A data frame with predictor variables for prediction.

Details

The function `lm_ac` fits a linear regression model using pairwise deletion for missing values. Instead of removing entire rows with missing values (as in complete-case analysis), this approach computes each component of the normal equations ($X'X$ and $X'y$) using the average of all available pairs.

`ac_mul` returns a symmetric matrix of average products between all pairs of columns.

`ac_vec` returns a vector of average products between each column of `X` and `y`.

`summary.lm_ac` prints the estimated coefficients.

`predict.lm_ac` generates predictions using the fitted model on new data.

Value

`lm_ac` returns an object of class `lm_ac`, a list containing:

<code>data</code>	Cleaned input data used in modeling.
<code>yName</code>	Response variable name.
<code>formula</code>	The model formula.

`fit_obj` A list with components `coef` (estimated coefficients) and `colnames` (predictor names).

`ac_mul` returns a numeric matrix.

`ac_vec` returns a numeric vector.

`summary.lm_ac` returns the coefficient vector (invisible).

`predict.lm_ac` returns a numeric vector of predictions.

Examples

```
data(airquality)

# Fit model
mod <- lm_ac(airquality, "Ozone")

# Print coefficients
summary(mod)

# Predict on new data
predict(mod, newdata = airquality)
```

lm_compare

Compare Missing-Data Methods with Bootstrap Summaries

Description

Run `bootstrap()` across one or more missing-data handling methods and collect the resulting cross-validation summary metrics in a single data frame.

Usage

```
lm_compare(
  data,
  predictors = NULL,
  yName,
  k = 5,
  task = c("regression", "classification"),
  methods = c("CC", "AC", "TOWER", "PREFILL"),
  seed = 42,
  impute_method = c("mice", "amelia", "missforest", "complete"),
  mice_method = NULL,
  m = 5,
  use_dummies = FALSE,
  tower_regFtnName = "lm",
  tower_opts = list(),
  tower_scaling = NULL,
  tower_yesYVal = NULL,
  ...
)
```

Arguments

<code>data</code>	A data frame containing predictors and the response variable.
<code>predictors</code>	Optional character vector of predictor names to keep. If <code>NULL</code> , all non-response columns are used.
<code>yName</code>	The name of the response column in <code>data</code> .
<code>k</code>	Number of folds used for cross-validation.
<code>task</code>	Whether to summarize regression or classification performance.
<code>methods</code>	Character vector of methods to evaluate. Supported values are "CC", "AC", "TOWER", and "PREFILL".
<code>seed</code>	Random seed used for reproducibility.
<code>impute_method</code>	Imputation method used when "PREFILL" is included in <code>methods</code> .
<code>mice_method</code>	Optional mice method specification passed to PREFILL runs.
<code>m</code>	Number of multiple imputations used by applicable PREFILL methods.
<code>use_dummies</code>	Logical indicating whether categorical variables should be converted to dummies for Amelia-based PREFILL runs.
<code>tower_regFtnName</code>	Regression routine passed to TOWER.
<code>tower_opts</code>	Optional list of TOWER options.
<code>tower_scaling</code>	Optional scaling object passed to TOWER.
<code>tower_yesYVal</code>	Optional flag passed to TOWER.
<code>...</code>	Additional arguments forwarded to <code>bootstrap()</code> .

Details

`lm_compare()` is intended for method comparison under cross-validation. Internally it calls `bootstrap()` with `test_split = 0`, so the reported summaries correspond to cross-validation results rather than final holdout-test performance.

Value

A data frame containing one row per requested method and the corresponding average performance metrics.

Examples

```
data("auto-mpg", package = "mvPred")

df <- auto_mpg
df$car_name <- NULL

for (nm in names(df)) {
  suppressWarnings(df[[nm]] <- as.numeric(df[[nm]]))
}

lm_compare(df, yName = "mpg", k = 3, task = "regression", methods = c("CC", "AC"))
```

Description

Fit linear models to data with missing values by first imputing missing data using multiple imputation methods, then fitting linear models to the imputed datasets. Supports multiple imputation methods including mice, Amelia, and missForest, as well as complete-case analysis.

Usage

```
lm_prefill(
  data,
  yName,
  impute_method = "mice",
  m = 5,
  use_dummies = FALSE,
  holdout = NULL,
  ...
)

## S3 method for class 'lm_prefill'
summary(object, ...)

## S3 method for class 'lm_prefill'
predict(object, newdata, type = "response", ...)
```

Arguments

data	A data frame containing the variables for modeling, possibly with missing values.
yName	A character string specifying the name of the response variable in data.
impute_method	Character string specifying the imputation method to use. One of "mice" (default), "amelia", "missforest", or "complete".
m	Integer specifying the number of multiple imputations to generate. This is only used for "mice" and "amelia" methods. Default is 5.
use_dummies	Logical indicating whether to convert categorical predictors to dummy variables before imputation. This is only relevant for "amelia".
holdout	Optional holdout specification used to define testing rows. This may be a logical vector of length nrow(data) or a numeric vector of row indices. If NULL, all rows are treated as training data.
...	Additional arguments passed to the imputation function (mice::mice, Amelia::amelia, or missForest::missForest) or to stats::lm for model fitting.
object	An object of class "lm_prefill" returned by lm_prefill().

newdata	A data frame containing new data for prediction. Rows with missing values in newdata are omitted before prediction.
type	Type of prediction; passed to <code>stats::predict.lm</code> . Default is "response".

Details

`lm_prefill` is a wrapper function that first imputes missing values in the dataset using one of several supported methods, then fits linear models to the imputed datasets.

Supported imputation methods:

- "mice": Multiple Imputation by Chained Equations via the **mice** package.
- "amelia": Multiple imputation using the **Amelia** package.
- "missforest": Nonparametric imputation using random forests via the **missForest** package.
- "complete": Complete-case analysis by omitting rows with missing values.

The `use_dummies` argument controls whether categorical variables are converted to dummy variables before imputation. If `use_dummies = TRUE`, the package **regtools** is required for dummy variable creation.

Additional arguments passed via `...` are forwarded to the underlying imputation or modeling functions as appropriate.

The returned object is of class "lm_prefill" and contains the imputed data, fitted models, and metadata.

`summary.lm_prefill` returns model summaries for each imputed dataset or a single summary for complete-case or missForest methods.

`predict.lm_prefill` returns predictions averaged across imputations for multiple imputation methods, or direct predictions for single imputation or complete-case models.

If `newdata` contains missing values, those rows are omitted with a warning before prediction.

Value

An object of class "lm_prefill" containing at least the following components:

data	Original input data.
yName	Response variable name.
formula	Model formula used for fitting.
imputed_data	Imputed dataset(s) or imputation object.
impute_method	Imputation method used.
fit_obj	Fitted linear model(s).
missforest_OOBerror	If <code>impute_method = "missforest"</code> , the out-of-bag error estimate from missForest .

Author(s)

Cynthia Mascarenhas <cynmascarenhas@ucdavis.edu>

See Also

[mice](#), [amelia](#), [missForest](#), [lm](#), [predict.lm](#), [na.omit](#), [factorsToDummies](#)

Examples

```
library(mice)
library(Amelia)
library(missForest)

# Example dataset with missing values
data(airquality)
airquality$Ozone[1:10] <- NA

# Fit linear model with mice imputation
lm_obj <- lm_prefill(
  airquality,
  yName = "Ozone",
  impute_method = "mice",
  m = 5
)

# Summarize fitted models
summary(lm_obj)

# Predict on new data (complete cases only)
newdata <- airquality[11:20, ]
predict(lm_obj, newdata)

# Fit with Amelia and use dummy variables for categorical predictors
lm_obj2 <- lm_prefill(
  airquality,
  yName = "Ozone",
  impute_method = "amelia",
  m = 5,
  use_dummies = TRUE
)
summary(lm_obj2)
```

lm_tower*Fit Linear Models with Missing Data Using the Tower Method*

Description

lm_tower fits a linear regression model using the Tower Method implemented in the **toweranNA** package. This method handles missing data in both training and new datasets without explicit imputation by leveraging regression averaging.

Usage

```
lm_tower(data, yName, regFtnName = "lm", opts = list(), scaling = NULL, yesYVal = NULL)

## S3 method for class 'lm_tower'
summary(object, ...)

## S3 method for class 'lm_tower'
predict(object, newdata, ...)
```

Arguments

<code>data</code>	A data frame containing the training data with possible missing values.
<code>yName</code>	A character string specifying the name of the response variable in data.
<code>regFtnName</code>	A character string specifying the regression function to use (default is "lm").
<code>opts</code>	A list of options passed to <code>toweranNA::makeTower()</code> .
<code>scaling</code>	Optional scaling parameters passed to <code>toweranNA::makeTower()</code> .
<code>yesYVal</code>	Optional parameter passed to <code>toweranNA::makeTower()</code> .
<code>object</code>	An object of class "lm_tower" returned by <code>lm_tower()</code> .
<code>newdata</code>	A data frame containing new observations for prediction. May contain missing values.
<code>...</code>	Additional arguments (currently unused).

Details

The Tower Method implemented in the **toweranNA** package fits regression models that can handle missing data without explicit imputation. It uses regression averaging over subsets of complete cases to provide predictions even when new data contain missing values.

This function provides a convenient wrapper around `toweranNA::makeTower()` and associated prediction methods.

`lm_tower` is particularly useful when the primary goal is prediction rather than inference.

Value

`lm_tower` returns an object of class "lm_tower" containing the fitted Tower model and related information.

`summary.lm_tower` prints a brief summary of the fitted model, including the response variable, regression function, and number of complete cases used.

`predict.lm_tower` returns a numeric vector of predicted values for `newdata`. The Tower method handles missing values internally, so predictions are returned for all rows of `newdata`.

Author(s)

Cynthia Mascarenhas <cynmascarenhas@ucdavis.edu>

See Also

[makeTower](#), [predict.tower](#)

Examples

```
library(toweranNA)

data(airquality)
airquality$Ozone[1:10] <- NA
airquality$Solar.R[5:15] <- NA

# Fit the tower model
lm_tower_obj <- lm_tower(airquality, yName = "Ozone")

# Summarize the model
summary(lm_tower_obj)

# Prepare new data with missing values
newdata <- airquality[20:30, -which(names(airquality) == "Ozone")]
newdata[1, "Solar.R"] <- NA

# Predict on new data
preds <- predict(lm_tower_obj, newdata = newdata)
print(preds)
```

 qeMLna

Interface qeML Models with Missing-Value Preprocessing

Description

Apply a missing-value handling strategy before fitting a predictive model from the **qeML** package. The helper `addListElement()` appends or replaces an element in a list.

Usage

```
qeMLna(
  data,
  yName,
  qeMLftn,
  mvFtn,
  qeMLOpts = NULL,
  mvPredOpts = NULL,
  retainMVFTnOut = TRUE,
  seed = 9999,
  holdout = 1000
)

addListElement(l, newEltName, newElt)
```

Arguments

<code>data</code>	A data frame containing predictors and the response variable.
<code>yName</code>	The name of the response column in data.
<code>qeMLftn</code>	Character string naming the predictive function from qeML to call.
<code>mvFtn</code>	Character string naming the missing-value handling routine to apply first.
<code>qeMLOpts</code>	Optional list of arguments passed to the selected qeML function.
<code>mvPredOpts</code>	Optional list of arguments passed to the missing-value handling routine.
<code>retainMVftnOut</code>	Logical indicating whether to store the missing-value routine output in the returned object.
<code>seed</code>	Random seed passed through to applicable routines.
<code>holdout</code>	Holdout size used by the predictive routine if applicable.
<code>l</code>	A list to modify. If NULL, a new list is created.
<code>newEltName</code>	Name of the element to add or replace.
<code>newElt</code>	Value of the element to add or replace.

Value

`qeMLna()` returns an object of class "MVqeMLout" containing the **qeML** fit and, optionally, the missing-value handling output.

`addListElement()` returns the modified list.

Examples

```
data("english", package = "mvPred")

eng <- english[, c("age", "birth_order", "ethnicity", "sex", "mom_ed", "vocab")]
qeMLna(eng, "vocab", "qeLin", "compCases", retainMVftnOut = FALSE)

addListElement(list(a = 1), "b", 2)
```

Index

- * **models regression imputation missing**
 - lm_prefill, [9](#)
- * **regression missing-data prediction**
 - lm_tower, [11](#)
- ac_mul (lm_ac), [5](#)
- ac_vec (lm_ac), [5](#)
- addListElement (qeMLna), [13](#)
- amelia, [11](#)
- auto_mpg, [2](#)
- bootstrap, [3](#)
- compCases, [5](#)
- english (auto_mpg), [2](#)
- factorsToDummies, [11](#)
- lm, [11](#)
- lm_ac, [5](#)
- lm_compare, [7](#)
- lm_prefill, [9](#)
- lm_tower, [11](#)
- makeTower, [13](#)
- mice, [11](#)
- missForest, [11](#)
- na.omit, [11](#)
- NHkids (auto_mpg), [2](#)
- predict.lm, [11](#)
- predict.lm_ac (lm_ac), [5](#)
- predict.lm_prefill (lm_prefill), [9](#)
- predict.lm_tower (lm_tower), [11](#)
- predict.tower, [13](#)
- qeMLna, [13](#)
- summary.lm_ac (lm_ac), [5](#)
- summary.lm_prefill (lm_prefill), [9](#)
- summary.lm_tower (lm_tower), [11](#)